## Experiment 2: ALU Operations (ADD, SUB, MUL, DIV) in 8086 using MASM and Debug

```
.MODEL SMALL
.STACK 100H
.DATA
    NUM1  DB 09H
    NUM2  DB 03H
    SUMR  DW ?
    DIFF  DB ?
    PROD  DW ?
    QUOT  DB ?
    REMN  DB ?
.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    MOV AL,NUM1
    MOV AH,00H
    ADD AL,NUM2
    ADC AH,00H
    MOV SUMR,AX

    MOV AL,NUM1
    SUB AL,NUM2
    MOV DIFF,AL

    MOV AL,NUM1
    MUL NUM2
    MOV PROD,AX

    MOV AX,0000H
    MOV AL,NUM1
    DIV NUM2
    MOV QUOT,AL
    MOV REMN,AH

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

## Experiment 3: Number Conversion (HEX to BCD) using MASM

```
.MODEL SMALL
.STACK 100H
.DATA
    HEXNUM DW 0FFFH
    MSG1   DB 'HEX number : 0FFFH',0DH,0AH,'$'
    MSG2   DB 'BCD (decimal) : $'
.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,MSG1
    MOV AH,09H
    INT 21H

    MOV AX,HEXNUM
    MOV BX,000AH
    MOV CX,0000H

NEXT:
    MOV DX,0000H
    DIV BX
    PUSH DX
    INC CX
    CMP AX,0000H
    JNE NEXT

    LEA DX,MSG2
    MOV AH,09H
    INT 21H

SHOW:
    POP DX
    ADD DL,30H
    MOV AH,02H
    INT 21H
    LOOP SHOW

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

## Experiment 4: String Operations in 8086 using DOS and BIOS Interrupts

```
.MODEL SMALL
.STACK 100H
.DATA
    MSG1 DB 'Enter a string : $'
    MSG2 DB 0DH,0AH,'Length   : $'
    MSG3 DB 0DH,0AH,'Reverse  : $'
    MSG4 DB 0DH,0AH,'String is a PALINDROME$'
    MSG5 DB 0DH,0AH,'String is NOT a palindrome$'
    BUF  DB 50
         DB ?
         DB 50 DUP(0)
.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX
    MOV ES,AX

    MOV AX,0003H
    INT 10H

    LEA DX,MSG1
    MOV AH,09H
    INT 21H

    LEA DX,BUF
    MOV AH,0AH
    INT 21H

    LEA DX,MSG2
    MOV AH,09H
    INT 21H
    MOV AL,BUF+1
    MOV AH,00H
    MOV BL,10
    DIV BL
    MOV BH,AH
    ADD AL,30H
    MOV DL,AL
    MOV AH,02H
    INT 21H
    ADD BH,30H
    MOV DL,BH
    MOV AH,02H
    INT 21H

    LEA DX,MSG3
    MOV AH,09H
    INT 21H
    MOV CL,BUF+1
    MOV CH,00H
    JCXZ DONE_REV
    LEA SI,BUF+2
    ADD SI,CX
    DEC SI
REV:
    MOV DL,[SI]
    MOV AH,02H
    INT 21H
    DEC SI
    LOOP REV
DONE_REV:

    MOV CL,BUF+1
    MOV CH,00H
    LEA SI,BUF+2
    LEA DI,BUF+2
    ADD DI,CX
    DEC DI
    SHR CX,1
    JCXZ ISPAL
CHK:
    MOV AL,[SI]
    CMP AL,[DI]
```

```
    JNE NOTPAL
    INC SI
    DEC DI
    LOOP CHK
ISPAL:
    LEA DX,MSG4
    JMP PRINT
NOTPAL:
    LEA DX,MSG5
PRINT:
    MOV AH,09H
    INT 21H

    MOV AH,00H
    INT 16H

    MOV AH,4CH
    INT 21H
MAIN ENDP
END MAIN
```

## Experiment 5: Addition of Two 8-bit BCD Numbers (Keyboard Input, INT 21H Output)

```
.MODEL SMALL
.STACK 100H
.DATA
    M1  DB 'Enter first 2-digit BCD number  : $'
    M2  DB 0DH,0AH,'Enter second 2-digit BCD number: $'
    M3  DB 0DH,0AH,'BCD Sum = $'
    N1  DB ?
    N2  DB ?
.CODE
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    LEA DX,M1
    MOV AH,09H
    INT 21H
    CALL READ_BCD
    MOV N1,AL

    LEA DX,M2
    MOV AH,09H
    INT 21H
    CALL READ_BCD
    MOV N2,AL

    MOV AL,N1
    ADD AL,N2
    DAA
    MOV BL,AL
    MOV BH,00H
    ADC BH,00H

    LEA DX,M3
    MOV AH,09H
    INT 21H

    CMP BH,00H
    JE  SKIP
    MOV DL,'1'
    MOV AH,02H
    INT 21H
SKIP:
    MOV AL,BL
    CALL PRINT_BCD

    MOV AH,4CH
    INT 21H
MAIN ENDP

READ_BCD PROC
    MOV AH,01H
    INT 21H
    SUB AL,30H
    MOV CL,04H
    SHL AL,CL
    MOV BL,AL
    MOV AH,01H
    INT 21H
    SUB AL,30H
    OR  AL,BL
    RET
READ_BCD ENDP

PRINT_BCD PROC
    MOV BL,AL
    MOV CL,04H
    SHR AL,CL
    ADD AL,30H
    MOV DL,AL
    MOV AH,02H
    INT 21H
    MOV AL,BL
    AND AL,0FH
```

```
    ADD AL,30H
    MOV DL,AL
    MOV AH,02H
    INT 21H
    RET
PRINT_BCD ENDP
END MAIN
```

## Experiment 6: Booth's Multiplication Algorithm

```
#include <stdio.h>
#define N 8

void printbin(int x)
{
    int i;
    for (i = N - 1; i >= 0; i--)
        printf("%d", (x >> i) & 1);
}

int main()
{
    int multiplicand, multiplier;
    int A = 0, Q, M, Qn = 0, q0, i, result;
    char *op;

    printf("Enter multiplicand and multiplier (-128 to 127): ");
    scanf("%d %d", &multiplicand, &multiplier);

    M = multiplicand & 0xFF;
    Q = multiplier & 0xFF;

    printf("\nStep      A          Q       Q-1   Operation\n");
    printf("-------------------------------------------------\n");
    printf("Init  ");  printbin(A); printf("   "); printbin(Q);
    printf("    %d\n", Qn);

    for (i = 1; i <= N; i++)
    {
        q0 = Q & 1;
        if (q0 == 1 && Qn == 0) {
            A = (A - M) & 0xFF;
            op = "A = A - M, shift";
        } else if (q0 == 0 && Qn == 1) {
            A = (A + M) & 0xFF;
            op = "A = A + M, shift";
        } else {
            op = "shift only";
        }

        Qn = q0;
        Q  = ((Q >> 1) | ((A & 1) << 7)) & 0xFF;
        A  = (A >> 1) | (A & 0x80);

        printf("%d     ", i);
        printbin(A); printf("   "); printbin(Q);
        printf("    %d    %s\n", Qn, op);
    }

    result = (A << 8) | Q;
    if (result & 0x8000)
        result -= 65536;

    printf("\nProduct = %d x %d = %d\n", multiplicand, multiplier, result);
    return 0;
}
```

## Experiment 7: Division Algorithm (Restoring and Non-Restoring)

```
#include <stdio.h>
#define N 8

void printbin(int x)
{
    int i;
    for (i = N; i >= 0; i--)
        printf("%d", (x >> i) & 1);
}

void restoring(int dividend, int divisor)
{
    int A = 0, Q = dividend, M = divisor, i;
    printf("\n--- RESTORING DIVISION ---\n");
    for (i = 1; i <= N; i++)
    {
        A = A * 2 + ((Q >> (N - 1)) & 1);
        Q = (Q << 1) & 0xFF;
        A = A - M;
        if (A < 0) {
            A = A + M;
            printf("Cycle %d: A=%3d Q=%3d  (A<0, restored, Q0=0)\n", i, A, Q);
        } else {
            Q = Q | 1;
            printf("Cycle %d: A=%3d Q=%3d  (A>=0, Q0=1)\n", i, A, Q);
        }
    }
    printf("Quotient = %d, Remainder = %d\n", Q, A);
}

void nonrestoring(int dividend, int divisor)
{
    int A = 0, Q = dividend, M = divisor, i, neg;
    printf("\n--- NON-RESTORING DIVISION ---\n");
    for (i = 1; i <= N; i++)
    {
        neg = (A < 0);
        A = A * 2 + ((Q >> (N - 1)) & 1);
        Q = (Q << 1) & 0xFF;
        if (neg) A = A + M;
        else     A = A - M;
        if (A >= 0) Q = Q | 1;
        printf("Cycle %d: A=%3d Q=%3d  (Q0=%d)\n", i, A, Q, Q & 1);
    }
    if (A < 0) A = A + M;
    printf("Quotient = %d, Remainder = %d\n", Q, A);
}

int main()
{
    int dividend, divisor;
    printf("Enter dividend and divisor (unsigned, max 255): ");
    scanf("%d %d", &dividend, &divisor);
    if (divisor == 0) {
        printf("Division by zero is not possible.\n");
        return 1;
    }
    restoring(dividend, divisor);
    nonrestoring(dividend, divisor);
    return 0;
}
```

## Experiment 8: Mapping Techniques of Cache Memory

```
#include <stdio.h>
#define LINES 4
#define WAYS  2
#define SETS  (LINES / WAYS)

int blocks[50], n;

void show(int c[], int size)
{
    int i;
    for (i = 0; i < size; i++) {
        if (c[i] == -1) printf("[ - ]");
        else            printf("[%2d ]", c[i]);
    }
    printf("\n");
}

void direct()
{
    int cache[LINES], i, idx, hits = 0;
    for (i = 0; i < LINES; i++) cache[i] = -1;
    printf("\n--- DIRECT MAPPING (line = block mod %d) ---\n", LINES);
    for (i = 0; i < n; i++) {
        idx = blocks[i] % LINES;
        if (cache[idx] == blocks[i]) {
            hits++;  printf("Block %2d -> HIT   ", blocks[i]);
        } else {
            cache[idx] = blocks[i];  printf("Block %2d -> MISS  ", blocks[i]);
        }
        show(cache, LINES);
    }
    printf("Hits = %d, Misses = %d, Hit ratio = %.2f\n", hits, n - hits, (float)hits / n);
}

void associative()
{
    int cache[LINES], i, j, found, ptr = 0, hits = 0;
    for (i = 0; i < LINES; i++) cache[i] = -1;
    printf("\n--- FULLY ASSOCIATIVE MAPPING (FIFO replacement) ---\n");
    for (i = 0; i < n; i++) {
        found = 0;
        for (j = 0; j < LINES; j++)
            if (cache[j] == blocks[i]) { found = 1; break; }
        if (found) {
            hits++;  printf("Block %2d -> HIT   ", blocks[i]);
        } else {
            cache[ptr] = blocks[i];
            ptr = (ptr + 1) % LINES;
            printf("Block %2d -> MISS  ", blocks[i]);
        }
        show(cache, LINES);
    }
    printf("Hits = %d, Misses = %d, Hit ratio = %.2f\n", hits, n - hits, (float)hits / n);
}

void setassoc()
{
    int cache[SETS][WAYS], ptr[SETS], i, j, s, found, hits = 0;
    for (i = 0; i < SETS; i++) {
        ptr[i] = 0;
        for (j = 0; j < WAYS; j++) cache[i][j] = -1;
    }
    printf("\n--- %d-WAY SET ASSOCIATIVE MAPPING (set = block mod %d, FIFO) ---\n", WAYS, SETS);
    for (i = 0; i < n; i++) {
        s = blocks[i] % SETS;
        found = 0;
        for (j = 0; j < WAYS; j++)
            if (cache[s][j] == blocks[i]) { found = 1; break; }
        if (found) {
            hits++;  printf("Block %2d -> HIT   ", blocks[i]);
        } else {
            cache[s][ptr[s]] = blocks[i];
            ptr[s] = (ptr[s] + 1) % WAYS;
```

```
            printf("Block %2d -> MISS  ", blocks[i]);
        }
        for (j = 0; j < SETS; j++) {
            printf("Set%d:", j);  show(cache[j], WAYS);
            if (j < SETS - 1) printf("                    ");
        }
    }
    printf("Hits = %d, Misses = %d, Hit ratio = %.2f\n", hits, n - hits, (float)hits / n);
}

int main()
{
    int i, ch;
    printf("Cache has %d lines.\nEnter number of block references: ", LINES);
    scanf("%d", &n);
    printf("Enter the block numbers: ");
    for (i = 0; i < n; i++) scanf("%d", &blocks[i]);
    do {
        printf("\n1. Direct  2. Associative  3. Set associative  4. Exit\nChoice: ");
        scanf("%d", &ch);
        if (ch == 1) direct();
        else if (ch == 2) associative();
        else if (ch == 3) setassoc();
    } while (ch != 4);
    return 0;
}
```

## Experiment 9: Drawing Basic Shapes using BIOS Services [Using C]

```
#include <stdio.h>
#include <conio.h>
#include <dos.h>
#include <math.h>

void setmode(int mode)
{
    union REGS r;
    r.h.ah = 0x00;
    r.h.al = mode;
    int86(0x10, &r, &r);
}

void putpixel_bios(int x, int y, int color)
{
    union REGS r;
    r.h.ah = 0x0C;
    r.h.al = color;
    r.h.bh = 0;
    r.x.cx = x;
    r.x.dx = y;
    int86(0x10, &r, &r);
}

void line(int x1, int y1, int x2, int y2, int color)
{
    int dx = x2 - x1, dy = y2 - y1, steps, i;
    float x = x1, y = y1, xinc, yinc;

    steps = (abs(dx) > abs(dy)) ? abs(dx) : abs(dy);
    if (steps == 0) { putpixel_bios(x1, y1, color); return; }
    xinc = (float)dx / steps;
    yinc = (float)dy / steps;
    for (i = 0; i <= steps; i++) {
        putpixel_bios((int)(x + 0.5), (int)(y + 0.5), color);
        x += xinc;
        y += yinc;
    }
}

void rectangle(int x1, int y1, int x2, int y2, int color)
{
    line(x1, y1, x2, y1, color);
    line(x2, y1, x2, y2, color);
    line(x2, y2, x1, y2, color);
    line(x1, y2, x1, y1, color);
}

void triangle(int x1, int y1, int x2, int y2, int x3, int y3, int color)
{
    line(x1, y1, x2, y2, color);
    line(x2, y2, x3, y3, color);
    line(x3, y3, x1, y1, color);
}

void main()
{
    clrscr();
    setmode(0x13);

    rectangle(20, 30, 140, 110, 4);
    triangle(220, 30, 170, 110, 270, 110, 2);
    line(20, 150, 300, 150, 14);

    getch();
    setmode(0x03);
}
```

## Experiment 10: File Operations using DOS Interrupts [in C]

```
#include <stdio.h>
#include <string.h>
#include <dos.h>
#include <conio.h>

int create_file(char *name)
{
    union REGS r;  struct SREGS s;
    r.h.ah = 0x3C;
    r.x.cx = 0;
    r.x.dx = FP_OFF(name);
    s.ds   = FP_SEG(name);
    intdosx(&r, &r, &s);
    if (r.x.cflag) return -1;
    return r.x.ax;
}

int open_file(char *name, int mode)
{
    union REGS r;  struct SREGS s;
    r.h.ah = 0x3D;
    r.h.al = mode;
    r.x.dx = FP_OFF(name);
    s.ds   = FP_SEG(name);
    intdosx(&r, &r, &s);
    if (r.x.cflag) return -1;
    return r.x.ax;
}

int write_file(int handle, char *buf, int len)
{
    union REGS r;  struct SREGS s;
    r.h.ah = 0x40;
    r.x.bx = handle;
    r.x.cx = len;
    r.x.dx = FP_OFF(buf);
    s.ds   = FP_SEG(buf);
    intdosx(&r, &r, &s);
    if (r.x.cflag) return -1;
    return r.x.ax;
}

int read_file(int handle, char *buf, int len)
{
    union REGS r;  struct SREGS s;
    r.h.ah = 0x3F;
    r.x.bx = handle;
    r.x.cx = len;
    r.x.dx = FP_OFF(buf);
    s.ds   = FP_SEG(buf);
    intdosx(&r, &r, &s);
    if (r.x.cflag) return -1;
    return r.x.ax;
}

void close_file(int handle)
{
    union REGS r;
    r.h.ah = 0x3E;
    r.x.bx = handle;
    intdos(&r, &r);
}

int delete_file(char *name)
{
    union REGS r;  struct SREGS s;
    r.h.ah = 0x41;
    r.x.dx = FP_OFF(name);
    s.ds   = FP_SEG(name);
    intdosx(&r, &r, &s);
    return r.x.cflag ? -1 : 0;
}
void main()
```

```
{
    char fname[] = "TEST.TXT";
    char text[]  = "COA practical: file handling using DOS interrupts.";
    char buf[100];
    int handle, n;

    clrscr();

    handle = create_file(fname);
    if (handle == -1) { printf("Cannot create file\n"); getch(); return; }
    n = write_file(handle, text, strlen(text));
    printf("File created, %d bytes written.\n", n);
    close_file(handle);

    handle = open_file(fname, 0);
    if (handle == -1) { printf("Cannot open file\n"); getch(); return; }
    n = read_file(handle, buf, 99);
    buf[n] = '\0';
    printf("File opened, %d bytes read.\nContents: %s\n", n, buf);
    close_file(handle);

    if (delete_file(fname) == 0)
        printf("File deleted successfully.\n");
    else
        printf("Could not delete file.\n");

    getch();
}
```